

outer optimization loop

`jax.grad / optax` over $\log \nu$ (one reverse-mode pass per step)

drives every method

Tesseract contract — one typed `apply` + `vector_jacobian_product` (where gradients are needed)

↓ `apply`

↑ VJP (gradient)

burgers_solver

differentiable spectral
Burgers solver (JAX)

VJP = PDE adjoint

PINN

`pinn_jax` $\rightleftharpoons$ `pinn_pytorch`

VJP = native autograd
(`jax.grad` / `torch`)

fmpe_posterior

amortized flow posterior

apply-only — no VJP
→ posterior ($\nu, A_{IC}, \varphi_{IC}$)

*Each component is a versioned, framework-agnostic container behind the same interface,
so switching method or backend is a one-line `Tesseract.from_image(...)` change.*